HELIX

FDE Project Timeline & Execution Plan

Objective: Build one application from scratch and progressively turn it into a cloud-native, observable, AI-powered, customer-facing system.

Recommended pace: ~10–12 hours/week. Estimated total: ~16–20 weeks. If you have more time, compress the schedule; do not skip the milestones.

Architecture Evolution

MVP → Docker → CI/CD → Cloud → Kubernetes → Observability → AI Agent → Multi-Agent/Supervisor → Customer Deployment → Production Hardening

16-Week Timeline

Weeks	Phase	Main Deliverable	Learn when needed
1–2	MVP foundation	React + FastAPI + PostgreSQL working end-to-end	HTTP, REST, TypeScript basics, Python/FastAPI, SQL
3	Authentication & core UX	Users, login, tasks, assignment, status, priority	JWT, auth, validation, API errors
4	Testing & code quality	Unit/API tests + clean repo structure	pytest, frontend tests, Git workflow
5	Docker	Full stack runs with docker compose	Images, Dockerfile, networking, volumes
6	CI/CD	Push → test → build image → registry	GitHub Actions, registries, secrets
7	Cloud deployment	Helix running on a cloud environment	DNS, HTTPS, networking, compute
8–9	Kubernetes	Helix deployed with Deployments/Services/Ingress	Pods, Services, ConfigMaps, Secrets, storage, rollouts
10	Reliability exercises	Scale, rollback, failure recovery	Health probes, resources, HPA, debugging
11	Observability	Dashboards + logs + metrics + traces	Prometheus, Grafana, OpenTelemetry
12–13	AI assistant	Natural-language task operations via tools	LLM APIs, tool calling, RAG only if needed
14	Agent architecture	Supervisor routes work to specialized agents	Agent orchestration, permissions, state
15	Customer scenario	Fictional customer requirements + deployment	Discovery, requirements, solution design, documentation
16+	Production hardening	Security, backups, alerts, runbooks, demo	Threat modeling, reliability, cost, operations

How to Execute Each Phase

Weeks 1–2 — Build the MVP

■ Create GitHub repo and README.

■ Create React frontend.

■ Create FastAPI backend.

■ Connect PostgreSQL.

■ Implement task CRUD.

■ Connect frontend → API → database.

■ Run locally and document setup.

Done when: a user can log in and create, update, assign, filter, and complete tasks from the browser.

Week 3 — Authentication & UX

■ Add registration/login.

■ Add JWT-based authentication.

■ Add user/task ownership and permissions.

■ Improve API validation and error handling.

■ Build basic dashboard.

Done when: different users see only what they are authorized to see.

Week 4 — Testing

■ Add pytest API tests.

■ Test authentication and task CRUD.

■ Add basic frontend tests.

■ Add linting/formatting.

■ Create development branch + pull-request workflow.

Done when: changes can be made without blindly relying on manual testing.

Week 5 — Docker

■ Write Dockerfiles for frontend/backend.

■ Create docker-compose.yml.

■ Containerize PostgreSQL and Redis if used.

■ Configure environment variables.

■ Add health checks.

■ Make one command start the stack.

Done when: a fresh machine can run Helix with docker compose up.

Week 6 — CI/CD

■ GitHub Actions runs tests.

■ Build Docker images.

■ Push images to a container registry.

■ Create deployment workflow.

■ Store secrets safely.

Done when: a Git push automatically produces a tested image.

Week 7 — Cloud

■ Choose one cloud.

■ Deploy Helix.

■ Configure networking.

■ Add DNS and HTTPS.

■ Document architecture and costs.

Done when: Helix is reachable publicly and survives a restart.

Weeks 8–9 — Kubernetes

■ Create namespaces.

■ Create Deployments and Services.

■ Add ConfigMaps and Secrets.

■ Add Ingress.

■ Handle persistent storage.

■ Deploy PostgreSQL appropriately for the learning environment.

■ Practice rollout/rollback and scaling.

Done when: the same Helix application runs on Kubernetes and you can explain every major resource.

Week 10 — Reliability

■ Add readiness/liveness probes.

■ Set resource requests/limits.

■ Scale replicas.

■ Simulate pod failure.

■ Simulate bad deployment and rollback.

■ Practice debugging with kubectl.

Done when: you can intentionally break Helix and recover it.

Week 11 — Observability

■ Centralize logs.

■ Expose application metrics.

■ Build Grafana dashboards.

■ Add tracing.

■ Create alerts for basic failure/latency conditions.

Done when: given 'Helix is slow', you can investigate instead of guessing.

Weeks 12–13 — AI Assistant

■ Define safe tools around Helix APIs.

■ Add LLM-powered natural-language interface.

■ Implement tool calling.

■ Allow task creation/search/update.

■ Add confirmation for risky actions.

■ Log agent actions.

Done when: 'Create a high-priority task for the backend team' results in a real API action.

Week 14 — Supervisor / Agents

■ Create a supervisor/router.

■ Create specialized Task, Analytics, and DevOps agents.

■ Define tool permissions.

■ Pass structured state between agents.

■ Trace agent decisions and tool calls.

Done when: one request can be routed to the appropriate specialized capability.

Week 15 — Customer Simulation

■ Invent a customer and business problem.

■ Write requirements.

■ Run a discovery-style interview (with yourself/peers).

■ Design the solution.

■ Deploy/configure Helix for the customer.

■ Create onboarding and troubleshooting docs.

Done when: you can present Helix as a solution to a business problem, not merely as a coding project.

Week 16+ — Production Hardening

■ Security review.

■ Backups/recovery.

■ Cost review.

■ Alerting.

■ Runbooks.

■ Architecture diagram.

■ Demo script.

■ Postmortem for a simulated incident.

Done when: you can explain how Helix is operated, secured, monitored, and supported.

Helix Repository Structure

```
helix/
■■■ frontend/ # React + TypeScript
■■■ backend/ # FastAPI application
■■■ worker/ # Background jobs
■■■ infra/
■ ■■■ docker/ # Dockerfiles / Compose
■ ■■■ kubernetes/ # K8s manifests
■ ■■■ terraform/ # Optional IaC later
■■■ agents/ # AI / supervisor logic
■■■ tests/
■■■ docs/
■ ■■■ architecture/
■ ■■■ runbooks/
■ ■■■ customer/
■■■ .github/workflows/
■■■ docker-compose.yml
■■■ README.md
```

Rules for the Journey

• Do not build throwaway tutorial projects unless a concept genuinely cannot be tested inside Helix.

• Every phase ends with a working artifact, not just notes.

• When something breaks, investigate it before reading the solution.

• Keep an engineering journal: problem → hypothesis → investigation → fix → lesson.

• Commit frequently and write useful commit messages.

• Document architecture decisions as you go.

• Do not add technology just because it is popular. Add it because Helix has a problem that requires it.

• Prioritize understanding the system end-to-end over mastering every individual tool.

Definition of Success

At the end of Helix, you should be able to take an ambiguous customer requirement, design the solution, build it, containerize it, deploy it to cloud/Kubernetes, observe and troubleshoot it, add AI-driven workflows, and explain the trade-offs to a technical or non-technical stakeholder.